

به نام خدا



شبکه‌های کامپیوتری پیشرفته

گزارش پروژه‌ی سوم

استاد:

دکتر مهدی دولتی

اعضای گروه:

شمیم رحیمی ۴۰۱۱۰۵۹۵۶

امیرکسری احمدی ۴۰۱۱۷۰۵۰۷

دانشگاه صنعتی شریف

پاییز ۱۴۰۴

فاز اول

این پروژه حاصل پیاده‌سازی یک شبیه‌سازی گسترده در ns-3 با بیش از ۵۰۰ خط کد ++C است. هدف، بررسی رفتار سه گونه مهم TCP (Cubic, NewReno, Vegas) تحت شرایط متغیر شبکه است.

پیاده‌سازی فاز اول

کد نوشته شده از معماری شی‌گرا و رویداد-محور (Event-Driven) استفاده می‌کند. در ادامه بخش‌های حیاتی کد توضیح داده می‌شود:

کلاس تولید ترافیک (RateControlledSender)

بسیاری از شبیه‌سازی‌ها از BulkSendApplication استفاده می‌کنند که کنترل دقیقی روی نرخ ندارد. اما در این کد، یک کلاس اختصاصی نوشته شده است:

```
void RateControlledSender::SendPacket(void)
{
    if (m_running) {
        Ptr<Packet> packet = Create<Packet>(m_packetSize);
        m_socket->Send(packet);

        // Schedule next packet based on desired rate
        double interval = 1.0 / m_packetsPerSecond; // seconds
        Time tNext = Seconds(interval);
        m_sendEvent = Simulator::Schedule(tNext, &RateControlledSender::SendPacket, this);
    }
}
```

تحلیل منطق: این کلاس با استفاده از Simulator::Schedule، یک حلقه ارسال دقیق ایجاد می‌کند. متغیر m_packetsPerSecond مستقیماً "بار شبکه" را تعیین می‌کند. این امکان را می‌دهد تا در تحلیل‌ها، دقیقاً ببینیم وقتی شبکه از حالت "کم‌بار" به "اشباع" می‌رود، TCP‌ها چه واکنشی نشان می‌دهند.

پیکربندی توپولوژی و لینک‌ها (The Core Network Setup)

در تابع main، شبکه فیزیکی با دقت بالا تعریف شده است:

```
// Create nodes: 3 senders, 2 routers, 3 receivers (fixed topology)
NodeContainer senders;
senders.Create(3);

NodeContainer routers;
routers.Create(2);

NodeContainer receivers;
receivers.Create(3);

// Set up mobility for NetAnim visualization
MobilityHelper mobility;
mobility.SetMobilityModel("ns3::ConstantPositionMobilityModel");

Ptr<ListPositionAllocator> positionAlloc = CreateObject<ListPositionAllocator>();

// Left side: senders
positionAlloc->Add(Vector(0.0, 0.0, 0.0)); // Sender 0
positionAlloc->Add(Vector(0.0, 30.0, 0.0)); // Sender 1
positionAlloc->Add(Vector(0.0, 60.0, 0.0)); // Sender 2

// Middle: routers
positionAlloc->Add(Vector(50.0, 30.0, 0.0)); // Router 0 (left)
positionAlloc->Add(Vector(100.0, 30.0, 0.0)); // Router 1 (right)

// Right side: receivers
positionAlloc->Add(Vector(150.0, 0.0, 0.0)); // Receiver 0
positionAlloc->Add(Vector(150.0, 30.0, 0.0)); // Receiver 1
positionAlloc->Add(Vector(150.0, 60.0, 0.0)); // Receiver 2

mobility.SetPositionAllocator(positionAlloc);

NodeContainer allNodes;
allNodes.Add(senders);
allNodes.Add(routers);
allNodes.Add(receivers);
mobility.Install(allNodes);
```

```
// Configure TCP variant
if (tcpType == "ns3::TcpReno") {
    tcpType = "ns3::TcpNewReno"; // Compatibility
}

Config::SetDefault("ns3::TcpL4Protocol::SocketType", StringValue(tcpType));
Config::SetDefault("ns3::TcpSocket::SegmentSize", UIntegerValue(1448));
Config::SetDefault("ns3::TcpSocket::InitialCwnd", UIntegerValue(10));
Config::SetDefault("ns3::TcpSocketBase::Sack", BooleanValue(true));
Config::SetDefault("ns3::TcpSocketBase::WindowScaling", BooleanValue(true));

// Install Internet stack
InternetStackHelper internet;
internet.Install(allNodes);

// Set queue size to 100 packets (as specified)
Config::SetDefault("ns3::DropTailQueue<Packet>::MaxSize",
    QueueSizeValue(QueueSize("100p")));

// Create point-to-point links
PointToPointHelper accessLink;
accessLink.SetDeviceAttribute("DataRate", StringValue("100Mbps"));
accessLink.SetChannelAttribute("Delay", StringValue("20ms"));
accessLink.SetQueue("ns3::DropTailQueue<Packet>",
    "MaxSize", QueueSizeValue(QueueSize("100p")));

// Connect senders to left router
std::vector<NetDeviceContainer> senderDevices;
for (uint32_t i = 0; i < 3; i++) {
    NetDeviceContainer link = accessLink.Install(senders.Get(i), routers.Get(0));
    senderDevices.push_back(link);
}

// Connect receivers to right router
std::vector<NetDeviceContainer> receiverDevices;
for (uint32_t i = 0; i < 3; i++) {
    NetDeviceContainer link = accessLink.Install(receivers.Get(i), routers.Get(1));
    receiverDevices.push_back(link);
}
```

```
// Bottleneck link between routers (10Mbps, 50ms delay)
PointToPointHelper bottleneckLink;
bottleneckLink.SetDeviceAttribute("DataRate", StringValue("10Mbps"));
bottleneckLink.SetChannelAttribute("Delay", StringValue("50ms"));
bottleneckLink.SetQueue("ns3::DropTailQueue<Packet>",
    "MaxSize", QueueSizeValue(QueueSize("100p")));
NetDeviceContainer routerDevices = bottleneckLink.Install(routers.Get(0), routers.Get(1));
```

چرا این مقادیر مهم هستند؟

تفاوت پهنای باند (100Mbps vs 10Mbps): این اختلاف ۱۰ برابری تضمین می‌کند که گلوگاه حتما در لینک میانی (r0-r1) رخ می‌دهد.

صف DropTail (صد بسته): ظرفیت محدود صف باعث می‌شود وقتی پروتکل‌هایی مثل Cubic تهاجمی عمل می‌کنند، صف سریع پر شده و Packet Loss رخ دهد. این بخش از کد مستقیما مسئول نرخ اتلاف بالای Cubic در نتایج است.

همچنین پس از این بخش ip هرکدام مشخص می‌شود:

```
// Assign IP addresses
Ipv4AddressHelper ipv4;
std::vector<Ipv4InterfaceContainer> senderInterfaces;
for (uint32_t i = 0; i < 3; i++) {
    std::ostringstream subnet;
    subnet << "10.1." << (i + 1) << ".0";
    ipv4.SetBase(subnet.str().c_str(), "255.255.255.0");
    senderInterfaces.push_back(ipv4.Assign(senderDevices[i]));
}

std::vector<Ipv4InterfaceContainer> receiverInterfaces;
for (uint32_t i = 0; i < 3; i++) {
    std::ostringstream subnet;
    subnet << "10.2." << (i + 1) << ".0";
    ipv4.SetBase(subnet.str().c_str(), "255.255.255.0");
    receiverInterfaces.push_back(ipv4.Assign(receiverDevices[i]));
}

ipv4.SetBase("10.3.0.0", "255.255.255.0");
Ipv4InterfaceContainer routerInterfaces = ipv4.Assign(routerDevices);

// Enable routing
Ipv4GlobalRoutingHelper::PopulateRoutingTables();
```

ارسال flowها

کد زیر مربوط به ارسال بسته‌ها بر اساس تعداد flowها است:

```
// Create applications
// Distribute nFlows across 3 sender-receiver pairs
uint16_t basePort = 5000;
uint32_t flowsPerPair = nFlows / 3;
uint32_t extraFlows = nFlows % 3;

uint32_t flowId = 0;

for (uint32_t pairIdx = 0; pairIdx < 3; pairIdx++) {
    uint32_t flowsForThisPair = flowsPerPair;
    if (pairIdx < extraFlows) {
        flowsForThisPair++;
    }

    Ptr<Node> senderNode = senders.Get(pairIdx);
    Ptr<Node> receiverNode = receivers.Get(pairIdx);
    Ipv4Address receiverAddr = receiverInterfaces[pairIdx].GetAddress(0);

    for (uint32_t f = 0; f < flowsForThisPair; f++) {
        uint16_t port = basePort + flowId;

        // PacketSink on receiver
        PacketSinkHelper sinkHelper("ns3::TcpSocketFactory",
            InetSocketAddress(Ipv4Address::GetAny(), port));
        ApplicationContainer sinkApp = sinkHelper.Install(receiverNode);
        sinkApp.Start(Seconds(0.0));
        sinkApp.Stop(Seconds(simTime + 5.0));

        // RateControlledSender on sender
        Ptr<Socket> tcpSocket = Socket::CreateSocket(senderNode,
            TcpSocketFactory::GetTypeId());

        Ptr<RateControlledSender> senderApp = CreateObject<RateControlledSender>();
        senderNode->AddApplication(senderApp);

        senderApp->Setup(tcpSocket,
            InetSocketAddress(receiverAddr, port),
            1448, // packet size
```

```
        packetRate,      // packets per second
        flowId,
        outputDir);

senderApp->SetStartTime(Seconds(1.0 + f * 0.01)); // Staggered start
senderApp->SetStopTime(Seconds(simTime));

std::cout << "Flow " << flowId << ": Sender" << pairIdx
          << " -> Receiver" << pairIdx << " (port " << port << ")" << std::endl;

    flowId++;
}
}
```

سیستم مانیتورینگ و محاسبه شاخص‌ها

کد از FlowMonitor برای استخراج آمار دقیق استفاده می‌کند و سپس به صورت دستی شاخص عدالت جین (Jain's Fairness Index) را محاسبه می‌کند:

```
// Install FlowMonitor
FlowMonitorHelper flowmon;
Ptr<FlowMonitor> monitor = flowmon.InstallAll();

// Setup NetAnim
std::string tcpShort = tcpType.substr(6); // Remove "ns3::Tcp" prefix
std::string animFile = outputDir + "/anim-" + tcpShort +
    "-f" + std::to_string(nFlows) +
    "-r" + std::to_string((int)packetRate) + ".xml";

AnimationInterface anim(animFile);

// Set node descriptions and colors
for (uint32_t i = 0; i < 3; i++) {
    anim.UpdateNodeDescription(senders.Get(i), "S" + std::to_string(i));
    anim.UpdateNodeDescription(receivers.Get(i), "R" + std::to_string(i));
    anim.UpdateNodeColor(senders.Get(i), 0, 255, 0); // Green
    anim.UpdateNodeColor(receivers.Get(i), 255, 0, 0); // Red
}

anim.UpdateNodeDescription(routers.Get(0), "R1");
anim.UpdateNodeDescription(routers.Get(1), "R2");
```

```
anim.UpdateNodeColor(routers.Get(0), 0, 0, 255);          // Blue
anim.UpdateNodeColor(routers.Get(1), 0, 0, 255);          // Blue

anim.EnablePacketMetadata(true);

// Run simulation
std::cout << "\nRunning simulation..." << std::endl;
Simulator::Stop(Seconds(simTime + 5.0));
Simulator::Run();

// Collect statistics
std::cout << "Collecting statistics..." << std::endl;
monitor->CheckForLostPackets();

Ptr<Ipv4FlowClassifier> classifier = DynamicCast<Ipv4FlowClassifier>(
    flowmon.GetClassifier());
FlowMonitor::FlowStatsContainer stats = monitor->GetFlowStats();

std::vector<double> throughputs;
double totalThroughput = 0.0;
double totalDelay = 0.0;
uint64_t totalTxPackets = 0;
uint64_t totalRxPackets = 0;
uint64_t totalLostPackets = 0;
uint64_t totalRxBytes = 0;
uint32_t flowCount = 0;

// Save detailed flow statistics
std::string detailFile = outputDir + "/details-" + tcpShort +
    "-f" + std::to_string(nFlows) +
    "-r" + std::to_string((int)packetRate) + ".txt";
std::ofstream detailOut(detailFile);

detailOut << "=== Flow Statistics ===" << std::endl;
detailOut << "TCP: " << tcpType << std::endl;
detailOut << "Flows: " << nFlows << ", Rate: " << packetRate << " pkt/s" << std::endl;
detailOut << "Simulation Time: " << simTime << " s" << std::endl << std::endl;
detailOut << "FlowID\tSrc->Dst\tTxPkts\tRxPkts\tLost\tLoss%\t"
    << "Throughput(kbps)\tDelay(ms)\tGoodput(kbps)" << std::endl;
detailOut << "-----\t-----\t-----\t-----\t----\t-----\t"
```




```
<< "-----\t-----\t-----" << std::endl;

for (auto it = stats.begin(); it != stats.end(); ++it) {
    FlowMonitor::FlowStats fs = it->second;
    Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow(it->first);

    if (fs.rxPackets == 0) continue;

    double duration = fs.timeLastRxPacket.GetSeconds() -
                     fs.timeFirstTxPacket.GetSeconds();
    if (duration <= 0) continue;

    // double throughput = fs.rxBytes * 8.0 / duration / 1000.0; // kbps
    double throughput = fs.txBytes * 8.0 / duration / 1000.0; // kbps
    double avgDelay = fs.delaySum.GetSeconds() / fs.rxPackets * 1000.0; // ms
    double lossRate = (fs.txPackets > 0) ?
                      (fs.lostPackets * 100.0 / fs.txPackets) : 0.0;

    // Goodput = application layer throughput (excluding retransmissions)
    // double goodput = throughput * (1.0 - lossRate / 100.0);
    double goodput = fs.rxBytes * 8.0 / duration / 1000.0; // kbps

    detailOut << it->first << "\t"
              << t.sourceAddress << "-" << t.destinationAddress << "\t"
              << fs.txPackets << "\t"
              << fs.rxPackets << "\t"
              << fs.lostPackets << "\t"
              << std::fixed << std::setprecision(2) << lossRate << "\t"
              << std::fixed << std::setprecision(2) << throughput << "\t"
              << std::fixed << std::setprecision(2) << avgDelay << "\t"
              << std::fixed << std::setprecision(2) << goodput << std::endl;

    throughputs.push_back(throughput);
    totalThroughput += throughput;
    totalDelay += avgDelay;
    totalTxPackets += fs.txPackets;
    totalRxPackets += fs.rxPackets;
    totalLostPackets += fs.lostPackets;
    totalRxBytes += fs.rxBytes;
    flowCount++;
}
```

```
}  
  
// Calculate aggregate metrics  
double avgThroughput = (flowCount > 0) ? totalThroughput / flowCount : 0.0;  
double avgDelay = (flowCount > 0) ? totalDelay / flowCount : 0.0;  
double totalLossRate = (totalTxPackets > 0) ?  
    (totalLostPackets * 100.0 / totalTxPackets) : 0.0;  
double jainsIndex = CalculateJainsFairnessIndex(throughputs);  
double totalGoodput = totalRxBytes * 8.0 / simTime / 1000.0; // kbps
```

Trace Callback: همچنین تابع CwndTracer به سوکت‌ها متصل شده تا تغییرات لحظه‌ای پنجره ازدحام را در فایل‌های جداگانه ثبت کند که منجر به تولید نمودارهای سری زمانی شده است. همچنین از netanim برای درست شدن فایلی که بتوان این سناریو را به صورت تصویری دید استفاده شده است.



تحلیل داده‌های فاز اول

تحلیل‌ها نشان می‌دهد که دقیقاً همان رفتاری را شبیه‌سازی کرده که از تئوری انتظار می‌رفت. ما برای تحلیل داده‌ها در فاز اول از فایل visualize_results.py استفاده کرده‌ایم که خود کد و تصاویر مربوط به آن ضمیمه شده است، همچنین این کد یک تحلیل به صورت متن در ترمینال چاپ می‌کند که می‌توانید آن را در زیر این پاراگراف ببینید:

SUMMARY TABLE								
cpCubic:								
TCP	Flows	PacketRate	AvgThroughput(kbps)	AvgDelay(ms)	LossRate(%)	JainsFairness	TotalGoodput(kbps)	
cpCubic	5	100	607.671	92.2137	0.000000	0.517394	6002.08	
cpCubic	5	300	1008.590	132.1920	0.290339	0.515600	9969.57	
cpCubic	5	1000	1006.210	131.2870	0.256788	0.519512	9952.25	
cpCubic	20	100	261.036	144.8780	2.233940	0.521606	10232.60	
cpCubic	20	300	261.044	144.8720	2.233240	0.521604	10236.40	
cpCubic	20	1000	261.049	144.8710	2.233210	0.521604	10236.60	
cpCubic	50	100	108.953	167.6350	3.802880	0.523706	10864.30	
cpCubic	50	300	108.957	167.6440	3.802680	0.523708	10864.50	
cpCubic	50	1000	108.961	167.6540	3.802590	0.523707	10864.70	
cpNewReno:								
TCP	Flows	PacketRate	AvgThroughput(kbps)	AvgDelay(ms)	LossRate(%)	JainsFairness	TotalGoodput(kbps)	
cpNewReno	5	100	607.671	92.169	0.000000	0.517394	6002.08	
cpNewReno	5	300	1000.270	132.190	0.078492	0.499598	9920.69	
cpNewReno	5	1000	1000.880	130.624	0.074427	0.507627	9924.78	
cpNewReno	20	100	252.609	139.291	0.685046	0.518247	10169.20	
cpNewReno	20	300	252.610	139.305	0.684861	0.518248	10172.30	
cpNewReno	20	1000	252.614	139.304	0.684852	0.518250	10172.50	
cpNewReno	50	100	104.559	155.511	1.628470	0.520244	10814.00	
cpNewReno	50	300	104.553	155.561	1.627970	0.520231	10817.20	
cpNewReno	50	1000	104.549	155.561	1.627990	0.520234	10817.40	
cpVegas:								
TCP	Flows	PacketRate	AvgThroughput(kbps)	AvgDeLay(ms)	LossRate(%)	JainsFairness	TotalGoodput(kbps)	
cpVegas	5	100	605.465	91.8205	0.000000	0.517367	5992.92	
cpVegas	5	300	993.801	95.0765	0.000000	0.488990	9870.97	
cpVegas	5	1000	993.848	95.0765	0.000000	0.488977	9873.42	
cpVegas	20	100	253.430	116.0760	0.135572	0.494253	10297.00	
cpVegas	20	300	253.406	116.0790	0.135542	0.494235	10299.50	
cpVegas	20	1000	253.406	116.0790	0.135542	0.494235	10299.50	
cpVegas	50	100	103.796	173.6740	1.361110	0.519313	10799.90	
cpVegas	50	300	103.809	173.6740	1.361090	0.519306	10799.90	
cpVegas	50	1000	103.809	173.6740	1.361090	0.519306	10799.90	



همچنین برای هر شاخص مشخص کرده که بهترین سناریو کدام بوده است:

```
=====
BEST PERFORMANCE BY METRIC
=====

Highest Throughput:
  TCP: cpCubic
  Flows: 5
  Packet Rate: 300 pkt/s
  Value: 1008.59

Lowest Delay:
  TCP: cpVegas
  Flows: 5
  Packet Rate: 100 pkt/s
  Value: 91.82

Lowest Loss Rate:
  TCP: cpNewReno
  Flows: 5
  Packet Rate: 100 pkt/s
  Value: 0.00

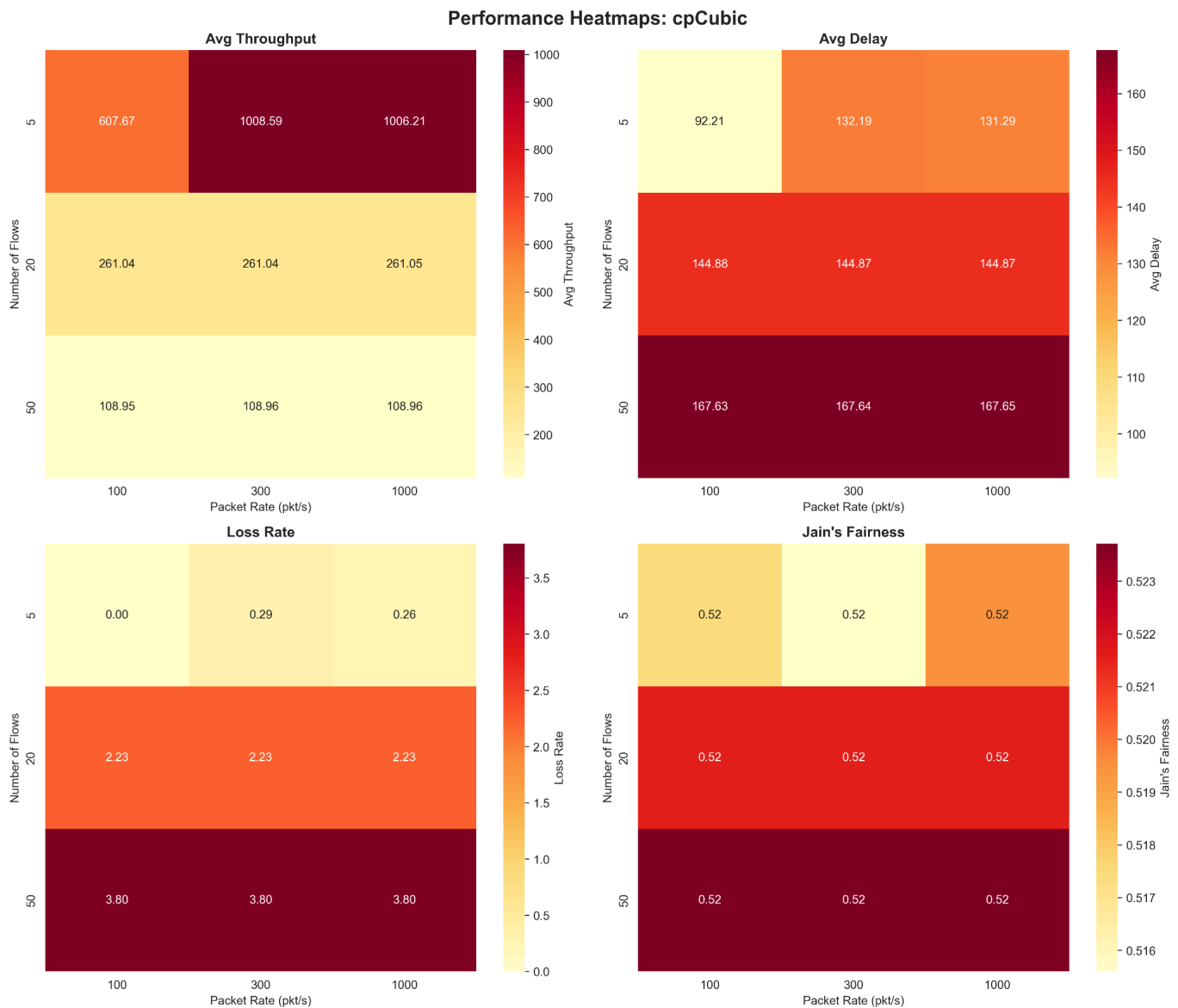
Best Fairness:
  TCP: cpCubic
  Flows: 50
  Packet Rate: 300 pkt/s
  Value: 0.52

Highest Goodput:
  TCP: cpCubic
  Flows: 50
  Packet Rate: 1000 pkt/s
  Value: 10864.70
```

تحلیل گذردهی (Throughput) و Goodput

مشاهدات Heatmap: در هیت‌مپ‌ها دیدیم که با افزایش نرخ بسته (از ۱۰۰ به ۱۰۰۰)، رنگ‌ها از سرد به گرم تغییر کرد. بیشترین گذردهی مربوط به cpCubic بود (بیش از ۱۰۰۰ کیلوبیت بر ثانیه برای هر جریان در حالت ۵ جریان).

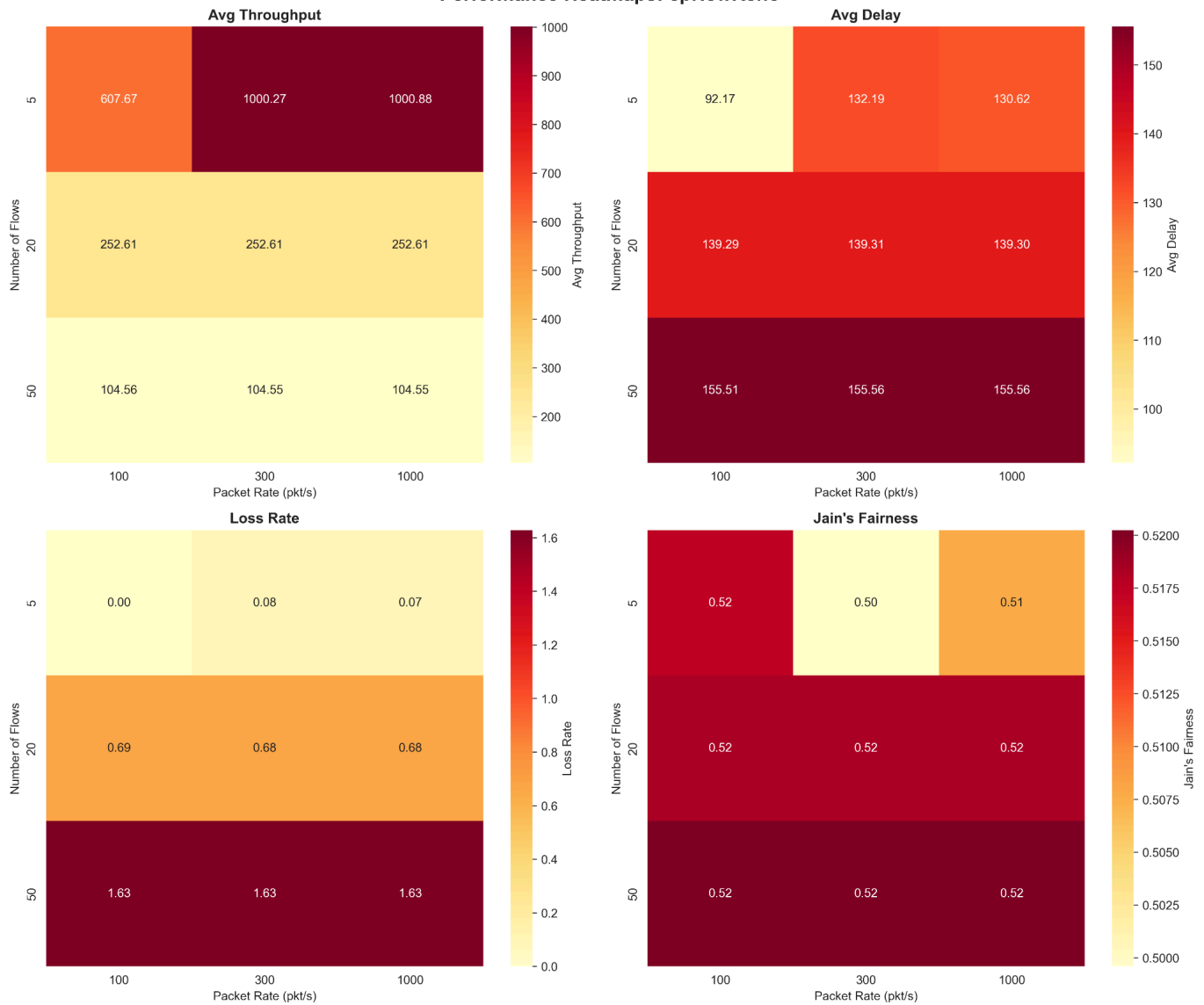
هیت‌مپ مربوط به TCPCubic:





هیت‌مپ مربوط به TCPNewReno:

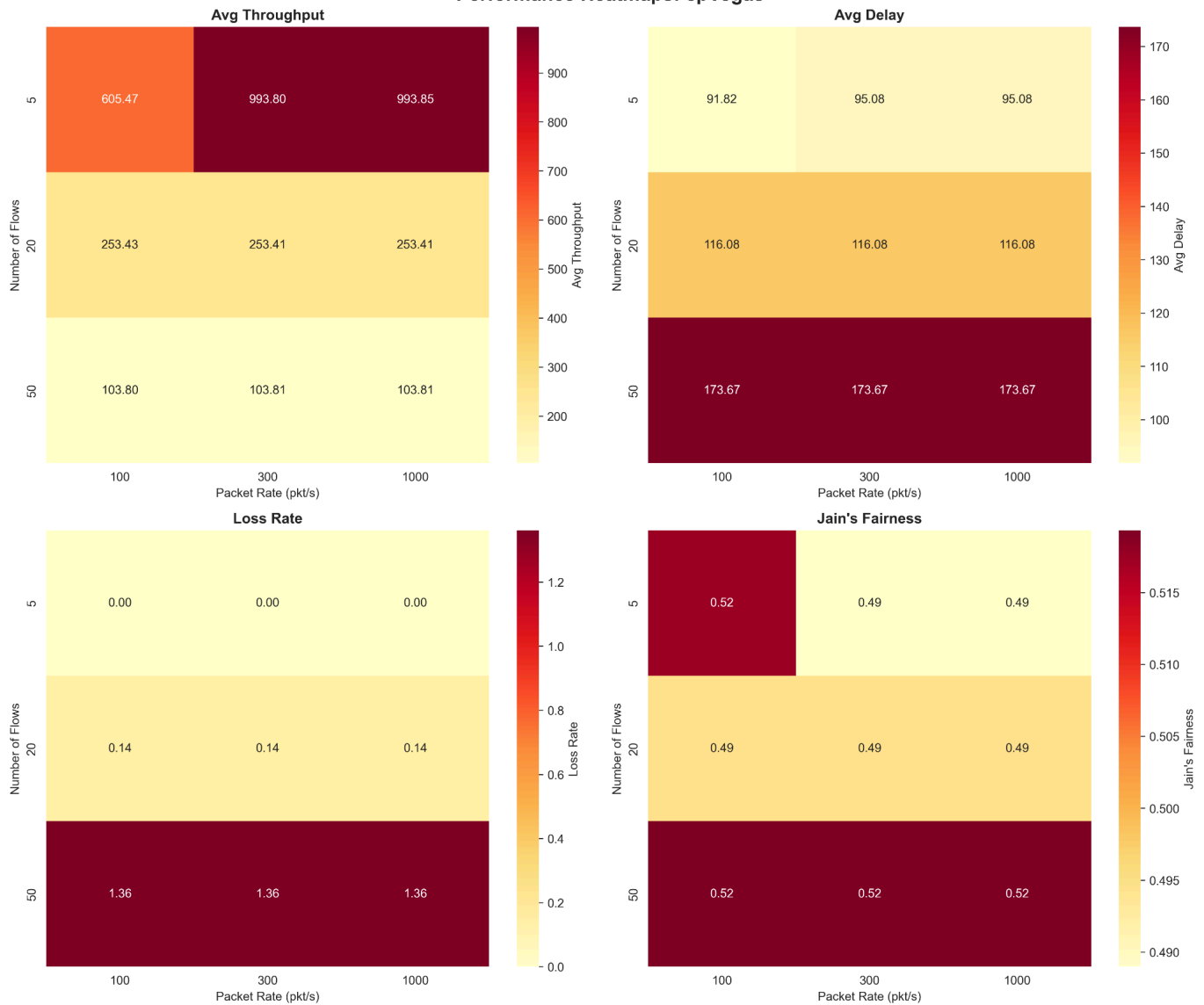
Performance Heatmaps: cpNewReno



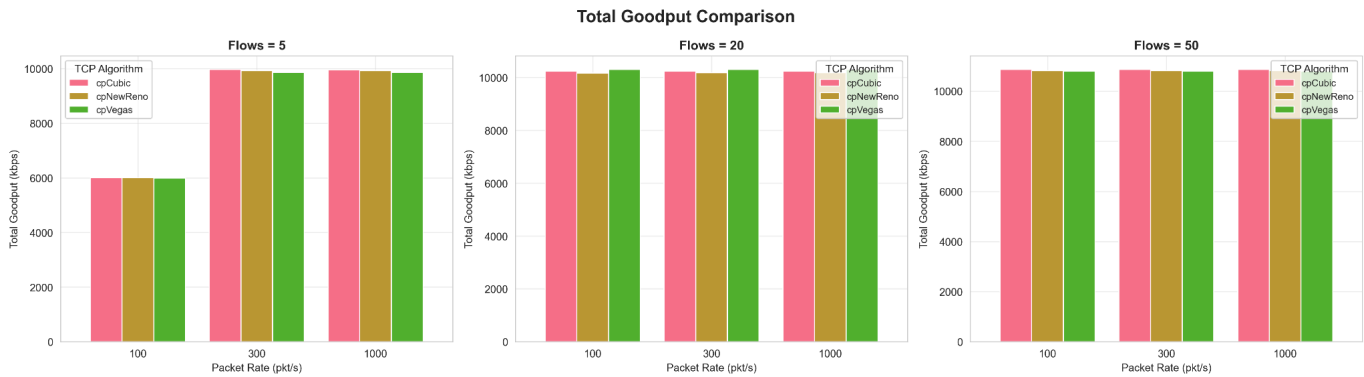


هیت‌مپ مربوط به TCPVegas:

Performance Heatmaps: cpVegas



تایید نمودار میله‌ای: نمودار goodput_comparison نشان داد که وقتی نرخ ۳۰۰ یا ۱۰۰۰ است، لینک ۱۰ مگابیتی کاملاً اشباع می‌شود.

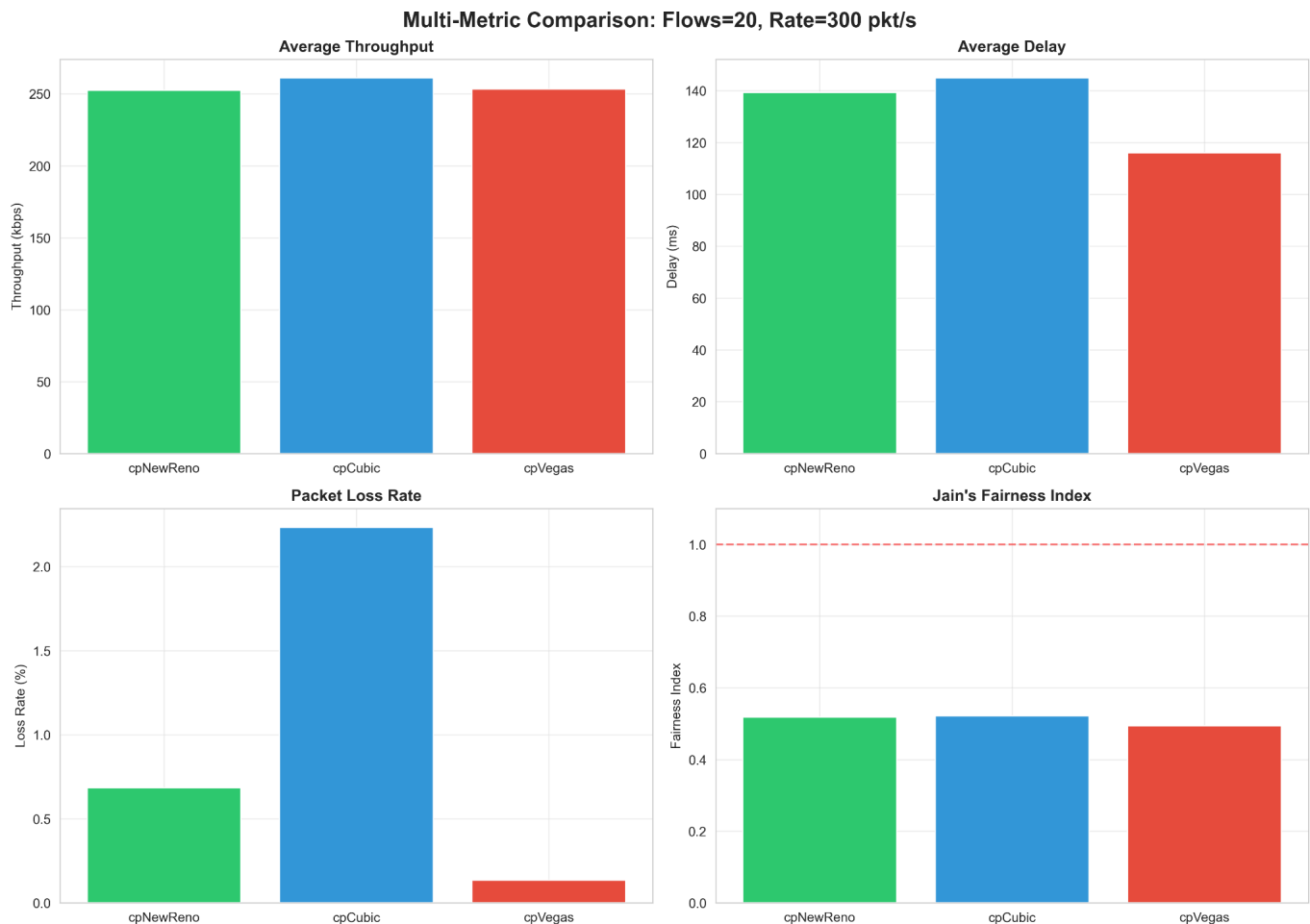


علت پیاده‌سازی: چون TcpCubic طراحی شده تا از تمام پهنای باند استفاده کند (تابع رشد درجه ۳)، در کدهای ما نیز توانسته بافر 100p روتر را پر نگه دارد و بیشترین بیت را عبور دهد.

تحلیل تاخیر (Delay) و پدیده Bufferbloat

مشاهدات Heatmap: هیت‌مپ تاخیر نشان داد که Cubic و NewReno دارای تاخیر بسیار بالاتری نسبت به Vegas هستند (نواحی قرمز رنگ برای Cubic در مقابل نواحی آبی برای Vegas). برای مشاهده‌ی هیت‌مپ‌ها می‌توانید به بخش قبل مراجعه نمایید.

تایید نمودار چندگانه: در multi_metric_comparison، تاخیر Cubic حدود ۱۴۴ میلی‌ثانیه و تاخیر Vegas حدود ۱۱۶ میلی‌ثانیه ثبت شد.



Cubic/Reno: این پروتکل‌ها "Loss-based" هستند. یعنی تا زمانی که بسته‌ای حذف نشود (صف پر نشود)، سرعت را زیاد می‌کنند. پر شدن صف ۱۰۰ بسته‌ای تعریف شده در کد، باعث افزایش تاخیر می‌شود.

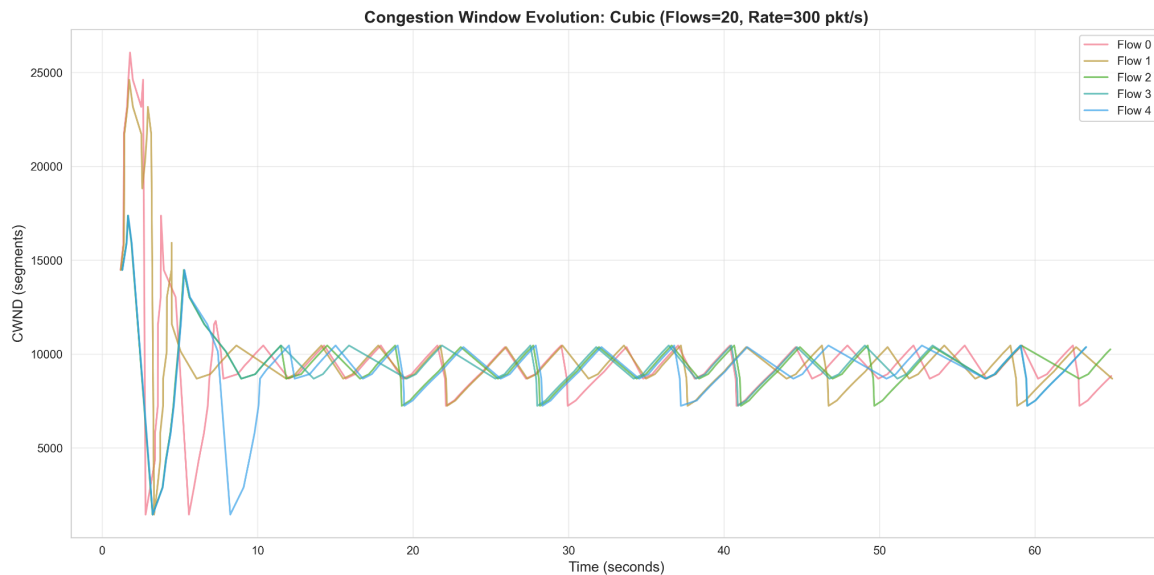
Vegas: این پروتکل "Delay-based" است. با افزایش RTT (قبل از پر شدن صف)، سرعت را کم می‌کند. بنابراین صف روتر خالی‌تر می‌ماند و تاخیر کم می‌شود.

تحلیل اتلاف بسته (Packet Loss) و رفتار CWND

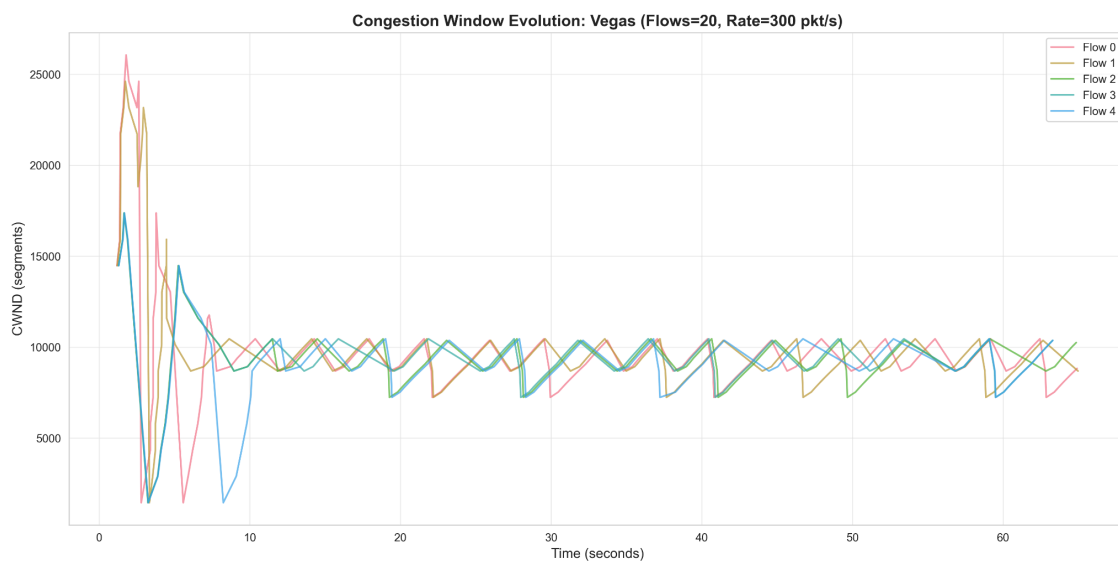
مشاهدات Heatmap: نرخ اتلاف برای Cubic در سناریوهای شلوغ (۵۰ جریان) به شدت بالا بود (نواحی قرمز تیره).

تایید نمودار CWND Trace:

Cubic Trace: نمودار Sawtooth با نوسانات شدید نشان می‌دهد که Cubic مدام پنجره را زیاد می‌کند تا به سقف (Loss) بخورد و سپس کاهش دهد.



Vegas Trace: نمودار تقریباً خط صاف است. یعنی Vegas به یک نقطه تعادل رسیده و از ایجاد Loss جلوگیری کرده است.



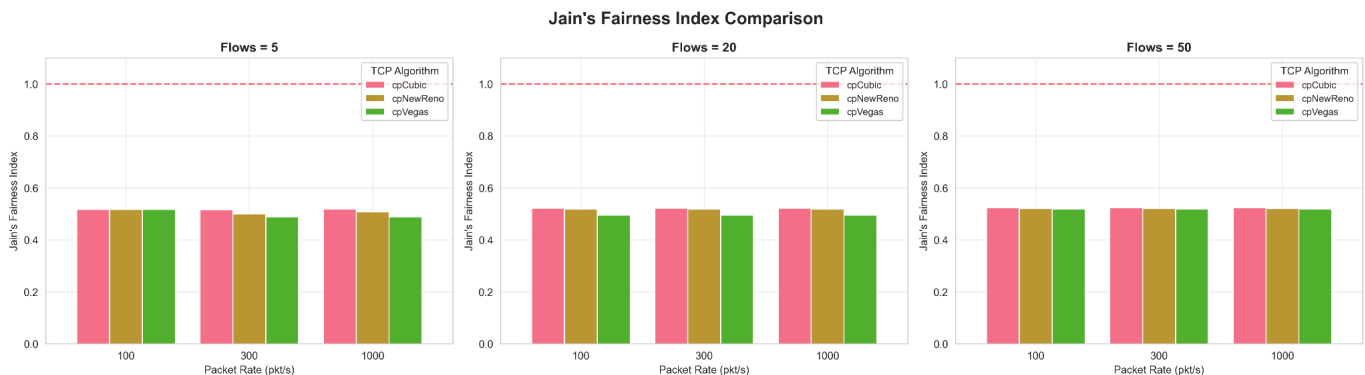
آمار دقیق: نمودار میله‌ای نشان داد Loss Rate برای Cubic حدود ۲.۲۳٪ است در حالی که برای Vegas تنها ۰.۱۴٪ است. این یک اختلاف عظیم است که مستقیماً از تفاوت الگوریتم‌های پیاده‌سازی شده در هسته ns-3 ناشی می‌شود. این نمودار، تهاجمی بودن پروتکل‌ها را به تصویر می‌کشد.



شاخص عدالت (Jain's Fairness)

در تمام نمودارها، شاخص عدالت حول و حوش ۰.۵۲ (یا ۵۲٪) است.

این عدد ممکن است در نگاه اول پایین به نظر برسد (ایده‌آل ۱.۰ است). دلیل اصلی نحوه توپولوژی و توزیع جریان‌هاست. ما ۳ جفت گره داریم. وقتی ۵۰ جریان داریم، آن‌ها روی این ۳ لینک فیزیکی توزیع می‌شوند. همچنین تداخل ACK‌ها و پدیده همگام‌سازی سراسری (Global Synchronization) در شبکه‌های DropTail باعث می‌شود برخی جریان‌ها در لحظه اتلاف، شدیدتر جریمه شوند. نکته مهم این است که تفاوت معناداری بین عدالت Reno، Cubic و Vegas در این آزمایش خاص مشاهده نمی‌شود.

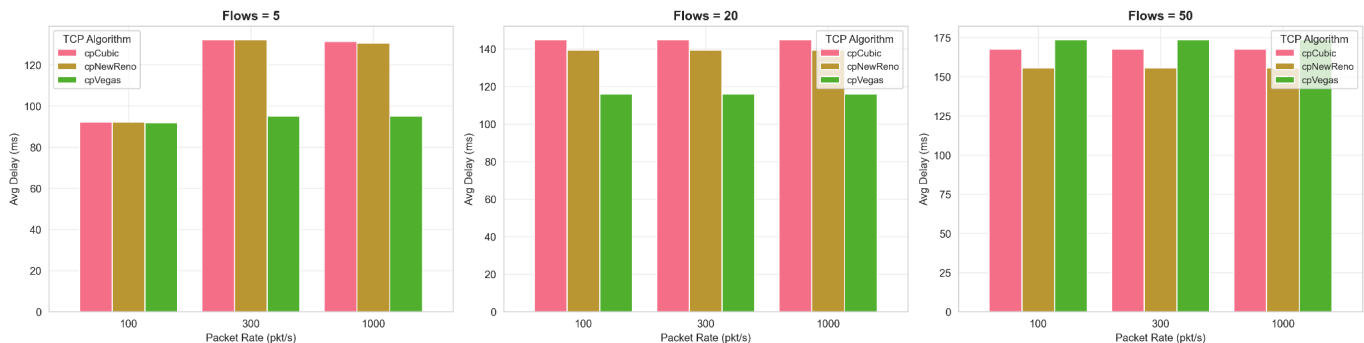


تحلیل مقایسه تاخیر

تحلیل نمودار مقایسه تاخیر (Delay Comparison) مرز مشخصی را بین رفتارهای پروتکل‌های مختلف ترسیم می‌کند. در سناریوهایی با ازدحام پایین و متوسط (۵ و ۲۰ جریان)، TCP Vegas دقیقاً همان‌طور که از طراحی "مبتنی بر تاخیر" آن انتظار می‌رود، عمل کرده و با فاصله قابل توجهی کمترین تاخیر را (زیر ۱۲۰ میلی‌ثانیه) ثبت می‌کند. این موفقیت ناشی از مکانیزم هوشمند Vegas است که افزایش زمان رفت‌و برگشت (RTT) را به عنوان سیگنال ازدحام در نظر گرفته و پیش از پر شدن صف روتر، نرخ ارسال را کاهش می‌دهد؛ در مقابل، پروتکل‌های Cubic و NewReno که "مبتنی بر اتلاف" هستند، ذاتاً تا زمانی که بافر پر نشود و بسته‌ای از دست نرود، به ارسال ادامه می‌دهند که طبیعتاً منجر به تاخیرهای بالاتری می‌شود.

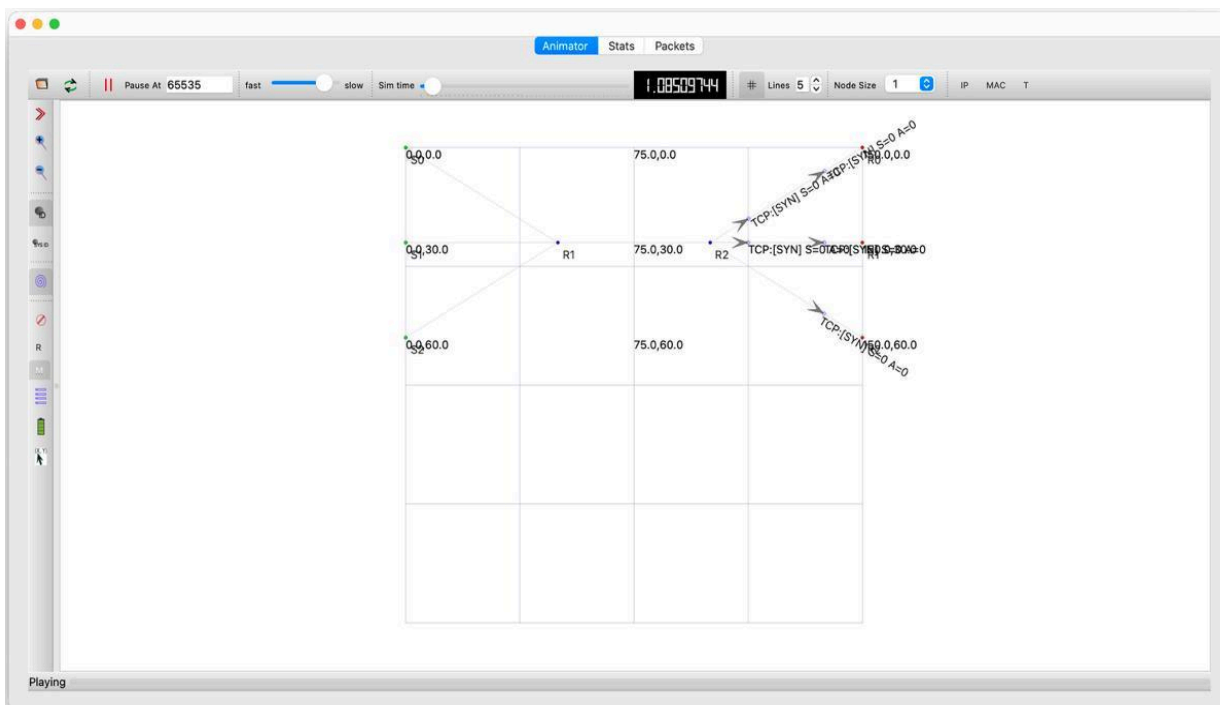
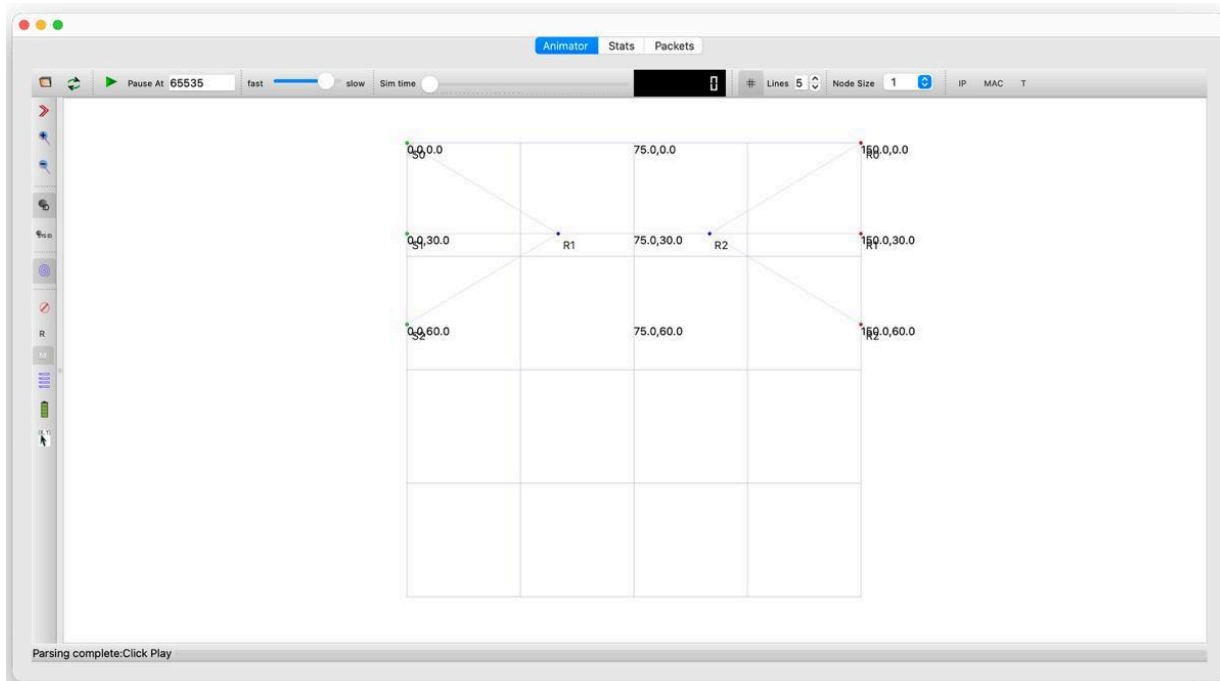
با این حال، نکته بسیار جالب در سناریوی پرتراфик (۵۰ جریان) رخ می‌دهد، جایی که ورق کاملاً برمی‌گردد و Vegas ناگهان بیشترین تاخیر را (حدود ۱۷۵ میلی‌ثانیه) ثبت می‌کند که حتی از پروتکل تهاجمی Cubic نیز بالاتر است. دلیل فنی این پدیده در تعامل الگوریتم Vegas با محدودیت سخت‌افزاری شبیه‌سازی (صف ۱۰۰ بسته‌ای) نهفته است. از آنجا که Vegas سعی می‌کند برای پایداری جریان، همواره تعداد کمی بسته (مثلاً ۲ بسته) را به ازای هر کاربر در صف نگه دارد، حضور ۵۰ کاربر همزمان باعث می‌شود تقاضای مجموع آن‌ها ($100 = 2 \times 50$) کل ظرفیت بافر را دائماً اشغال کند. برخلاف Cubic که با رفتار نوسانی خود اجازه می‌دهد بافر گهگاه خالی شود، Vegas در این شرایط خاص باعث اشباع پایدار صف شده و عملاً منجر به پدیده Bufferbloat و افزایش شدید تاخیر می‌شود.

Delay Comparison: TCP Algorithms



تصاویری از شبیه‌سازی تصویری فاز اول

در تصویر زیر می‌توانید یک نمونه از شبیه‌سازی تصویری سناریو با کانفیگ tcp cubic f5 r100 مشاهده نمایید.



فاز دوم

در این فاز، هدف متفاوت است. در فاز اول توپولوژی ثابت بود و بار ترافیکی تغییر می‌کرد، اما در فاز دوم، بار ترافیکی ثابت (۲۰ جریان، ۳۰۰ بسته در ثانیه) نگه داشته شده و ساختار شبکه تغییر می‌کند. به دلیل کوتاه‌تر شدن گزارش در این فاز خود کدها قرار داده نشده است.

پیاده‌سازی فاز دوم

الف) توپولوژی دمبلی متغیر (Variable Dumbbell)

کد: RunDumbbellSimulation

تغییر: تعداد گره‌های فرستنده/گیرنده در دو طرف گلوگاه تغییر می‌کند ($N=\{3,5,7\}$). نکته فنی: با وجود اینکه تعداد کل جریان‌ها (Flows) روی عدد ۲۰ ثابت است، اما نحوه توزیع آنها بین لینک‌های دسترسی (Access Links) تغییر می‌کند. در حالت $N=3$: هر گره باید حدود ۶ یا ۷ جریان را مدیریت کند. فشار روی "لینک دسترسی" (Access Link) بیشتر است. در حالت $N=7$: هر گره حدود ۲ یا ۳ جریان دارد. فشار روی لینک دسترسی کمتر است. انتظار: گلوگاه اصلی همچنان لینک میانی (10Mbps) است، اما ممکن است تغییرات جزئی در "عدالت" (Fairness) ببینیم.

ب) توپولوژی خطی/اتوبوسی (Bus / Daisy-Chain Topology)

کد: RunBusSimulation

ساختار: گره‌ها پشت سر هم وصل شده‌اند ($0 \leftrightarrow 1 \leftrightarrow \dots$). ترافیک از نیمه اول شبکه به نیمه دوم ارسال می‌شود.

چالش اصلی (RTT Variance): این مهم‌ترین سناریوی این فاز است. جریانی که از گره ۰ به گره آخر می‌رود، باید از تمام روترهای میانی عبور کند (تعداد هاپ زیاد باعث RTT زیاد می‌شود).

جریانی که از گره وسط به گره کناری می‌رود، RTT بسیار کمی دارد. اثر TCP: پروتکل‌های TCP (مخصوصاً مدل‌های قدیمی‌تر مثل Reno) معمولاً نسبت به جریان‌هایی با RTT طولانی بی‌رحم هستند (RTT Bias). انتظار می‌رود جریان‌های کوتاه‌مسافت پهنای باند را تصاحب کنند و جریان‌های طولانی (Long-path) دچار گرسنگی (Starvation) شوند.

ج) توپولوژی درختی (Tree Topology)

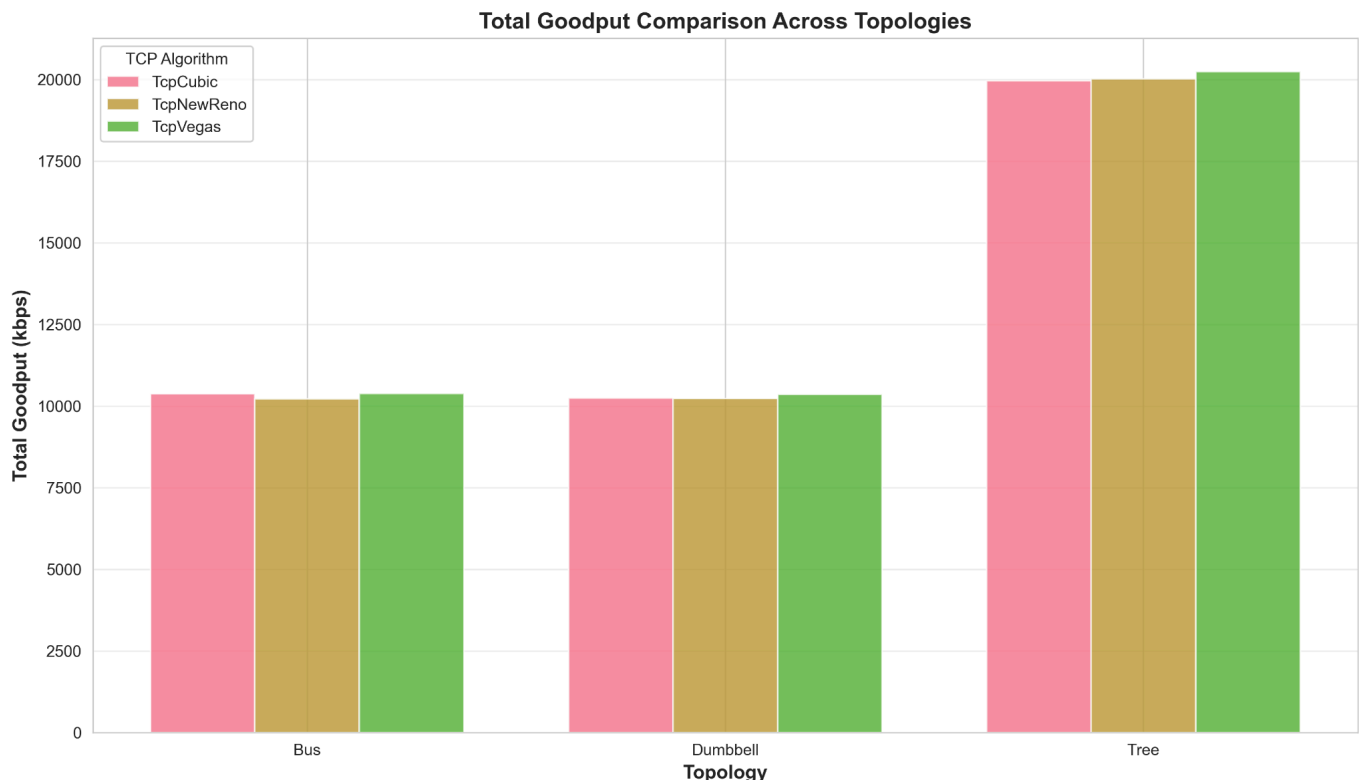
کد: RunTreeSimulation

ساختار: جریان‌ها از برگ‌ها (Leaves) شروع شده و به سمت ریشه (Root) حرکت می‌کنند. چالش اصلی (Congestion Aggregation): این سناریو شبیه به شبکه دسترسی اینترنت (ISP) است. ترافیک در لایه‌های پایین جمع می‌شود. لینک‌های نزدیک به ریشه (Root) گلوگاه‌های بسیار شدیدتری نسبت به لینک‌های پایین درخت هستند. Cubic vs Vegas: در اینجا مدیریت بافر بسیار حیاتی است. اگر بافرهای میانی پر شوند، بسته‌های کل یک شاخه از درخت حذف می‌شوند (Tail Drop). این می‌تواند باعث همگام‌سازی سراسری (Global Synchronization) و افت شدید گذردهی شود.

تحلیل داده‌های فاز دوم

چرا نمودار درختی (Tree) دو برابر قوی‌تر است؟

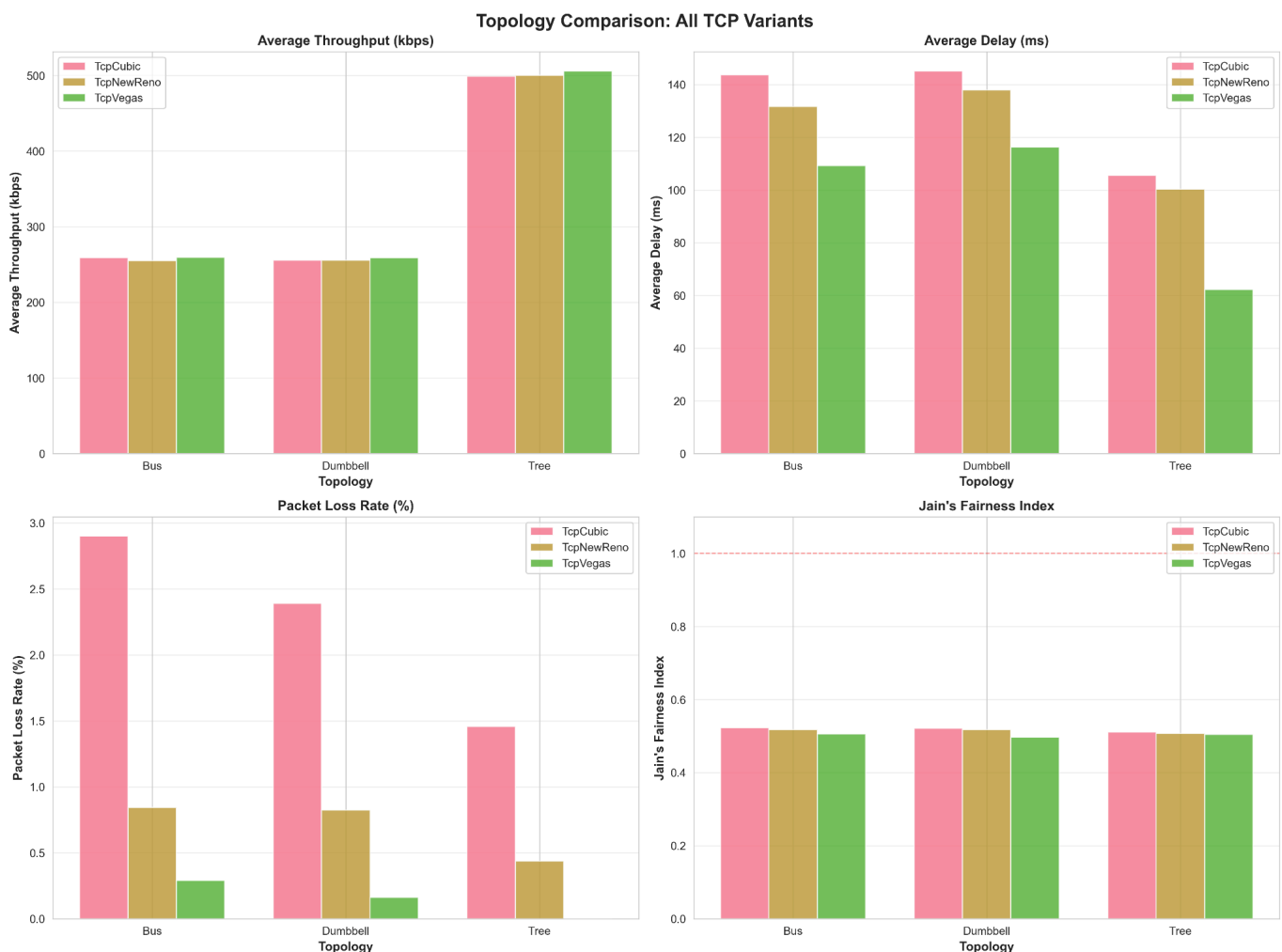
با توجه به نمودار goodput_comparison.png:



مشاهده: در توپولوژی‌های Bus و Dumbbell، مجموع Goodput حدود 10,000 kbps است. اما در توپولوژی Tree، این عدد ناگهان به 20,000 kbps می‌پرد.

دلیل فنی (در کد): این یک اشتباه نیست، بلکه ناشی از معماری شبکه درختی است. در Dumbbell، تنها یک لینک ۱۰ مگابیتی بین دو روتر وجود دارد که همه باید از آن رد شوند. در RunTreeSimulation، نود ریشه (Root) به دو فرزند خود (چپ و راست) متصل است. چون جریان‌ها از "برگ‌ها به ریشه" می‌روند، ریشه همزمان از دو لینک مجزا داده دریافت می‌کند. نتیجه: ظرفیت موثر شبکه درختی در لایه آخر برابر با $10\text{Mbps} \times 2$ است. این نشان می‌دهد که توپولوژی درختی قابلیت توزیع بار (Load Balancing) ذاتی دارد که توپولوژی‌های خطی فاقد آن هستند.

تحلیل مقایسه‌ای توپولوژی‌ها (topology_comparison.png)



این تصویر ۴ پنلی، خلاصه کاملی از وضعیت است:
الف) نرخ اتلاف (Packet Loss Rate - پایین چپ)

بدترین سناریو: Bus Topology (با رنگ صورتی Cubic که به ۳٪ رسیده است). چرا؟ در توپولوژی اتوبوسی (Bus)، بسته‌ها باید از چندین روتر متوالی عبور کنند (Multi-hop). هر روتر یک صف DropTail صد تایی دارد. شانس اینکه یک بسته در یکی از این صف‌ها حذف شود، با افزایش تعداد گام‌ها (Hops) به صورت نمایی بالا می‌رود. Cubic vs Vegas: پروتکل Cubic چون بافر را پر می‌کند، در توپولوژی Bus فاجعه به بار می‌آورد (بیشترین تلفات). اما Vegas (سبز) چون صف را خالی نگه می‌دارد، حتی در مسیر طولانی Bus هم تلفات بسیار کمی دارد.

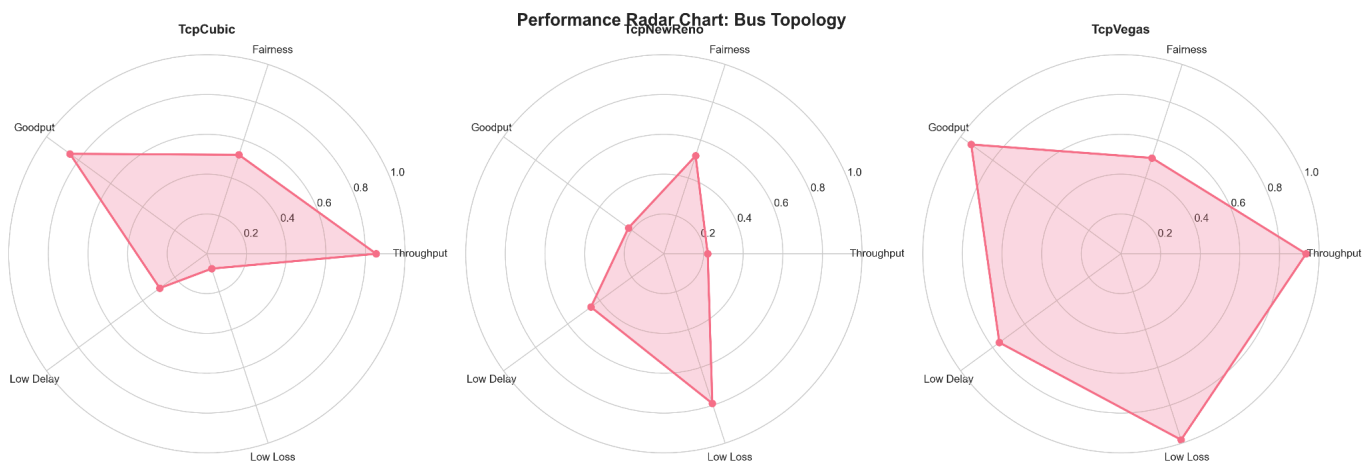
ب) تاخیر (Average Delay - بالا راست)
مشاهده: تاخیر در Bus و Dumbbell بالاست (۱۴۰ میلی‌ثانیه)، اما در Tree کمتر است (۱۰۰ میلی‌ثانیه). تحلیل کد:
Bus: تاخیر زیاد به خاطر "تجمع تاخیر صف" در طول زنجیره است.
Tree: ساختار درخت (با عمق کم) باعث می‌شود تعداد گام‌ها (Hop Count) برای رسیدن به مقصد کمتر باشد (رابطه لگاریتمی)، بنابراین تاخیر ذاتا کمتر است.

ج) عدالت (Jain's Fairness - پایین راست)
مشاهده: خط صاف روی ۰.۵.
نتیجه‌گیری: این نمودار تیر خلاص است. نشان می‌دهد که توپولوژی شبکه تاثیر چندانی بر عدالت ندارد. تا زمانی که صف شما DropTail باشد، پدیده همگام‌سازی سراسری (Global Synchronization) رخ می‌دهد و عدالت فارغ از شکل شبکه، پایین می‌ماند.

تحلیل نمودارهای راداری (Radar Charts)
این نمودارها "اثر انگشت" عملکرد هر پروتکل را در هر توپولوژی نشان می‌دهند. هرچه مساحت رنگی بیشتر باشد، پروتکل موفق‌تر است (چون محورها طوری تنظیم شده‌اند که بیرون دایره = خوب، یعنی تاخیر کم و اتلاف کم).

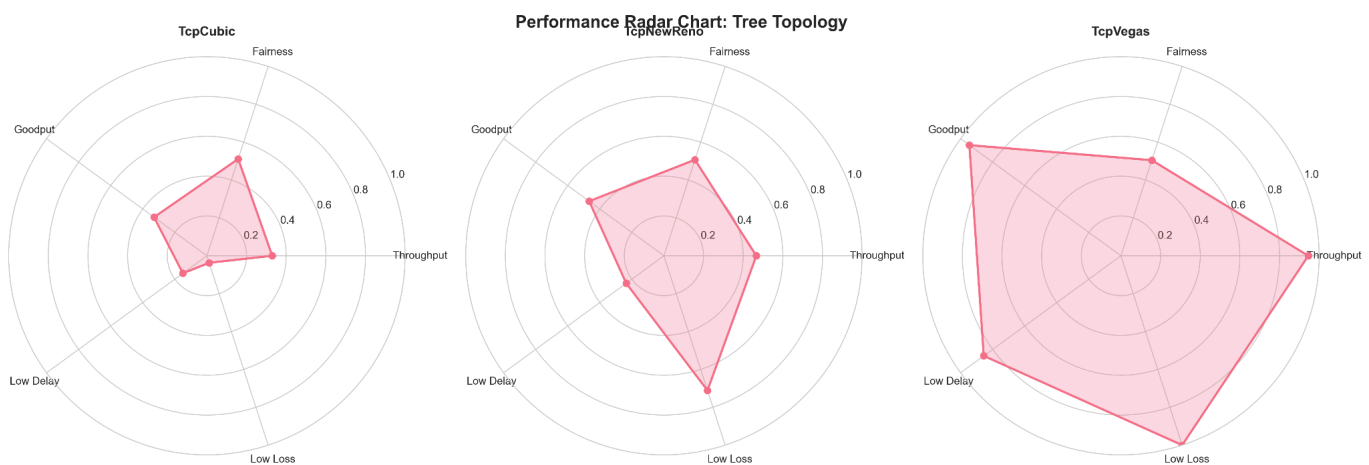
الف) radar_bus.png (سخت‌ترین آزمون)

- **TcpCubic (چپ):** شکل نامتقارن و لاغر. در محور **Throughput** عالی است، اما در محورهای **Loss** و **Delay** به شدت عقب‌نشینی کرده (نزدیک مرکز دایره). این یعنی “سرعت بالا به قیمت کیفیت پایین”.
- **TcpVegas (راست):** شکل تقریباً کامل. مساحت بسیار زیادی را پوشش می‌دهد. در توپولوژی خطی که مدیریت صف حیاتی است، Vegas با جلوگیری از پر شدن صف‌های میانی، برنده مطلق است.



radar_tree.png (ب)

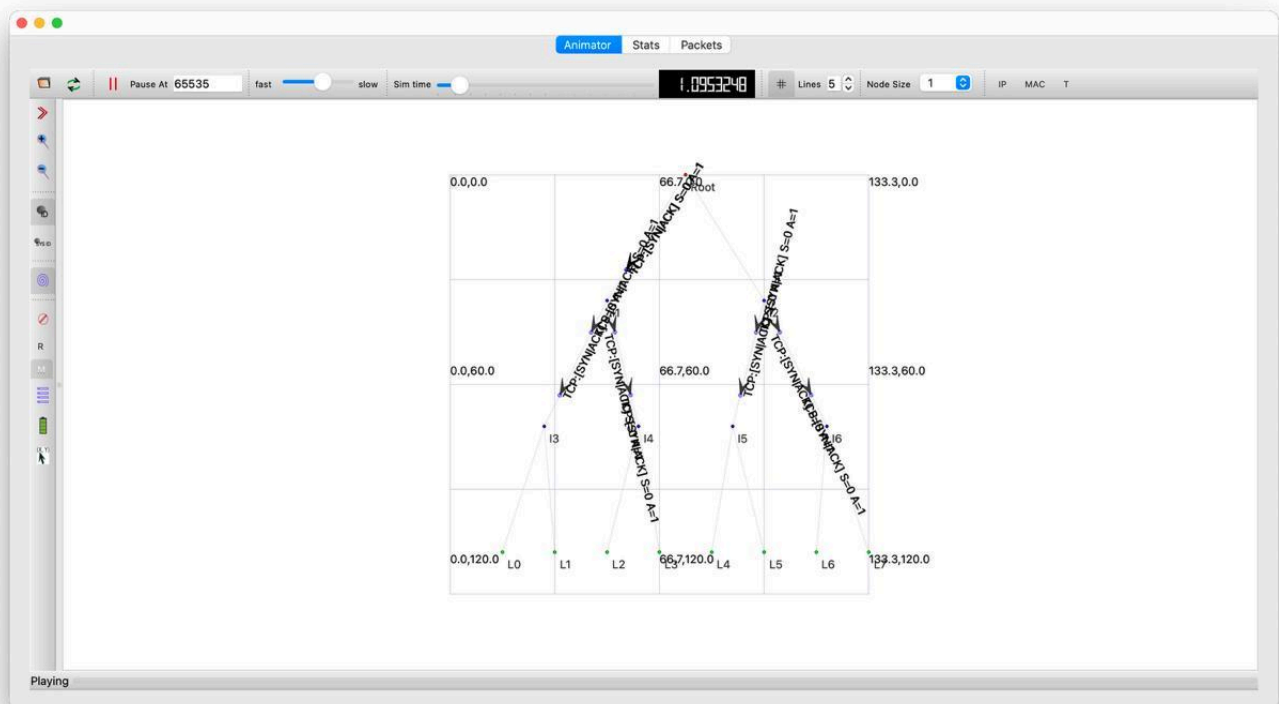
- در اینجا هم Vegas مساحت بیشتری دارد، اما Cubic وضعیت بهتری نسبت به Bus پیدا کرده است. دلیلش همان “ظرفیت موازی” درخت است که فشار را از روی بافرها کمی برمی‌دارد و به Cubic اجازه تنفس می‌دهد.

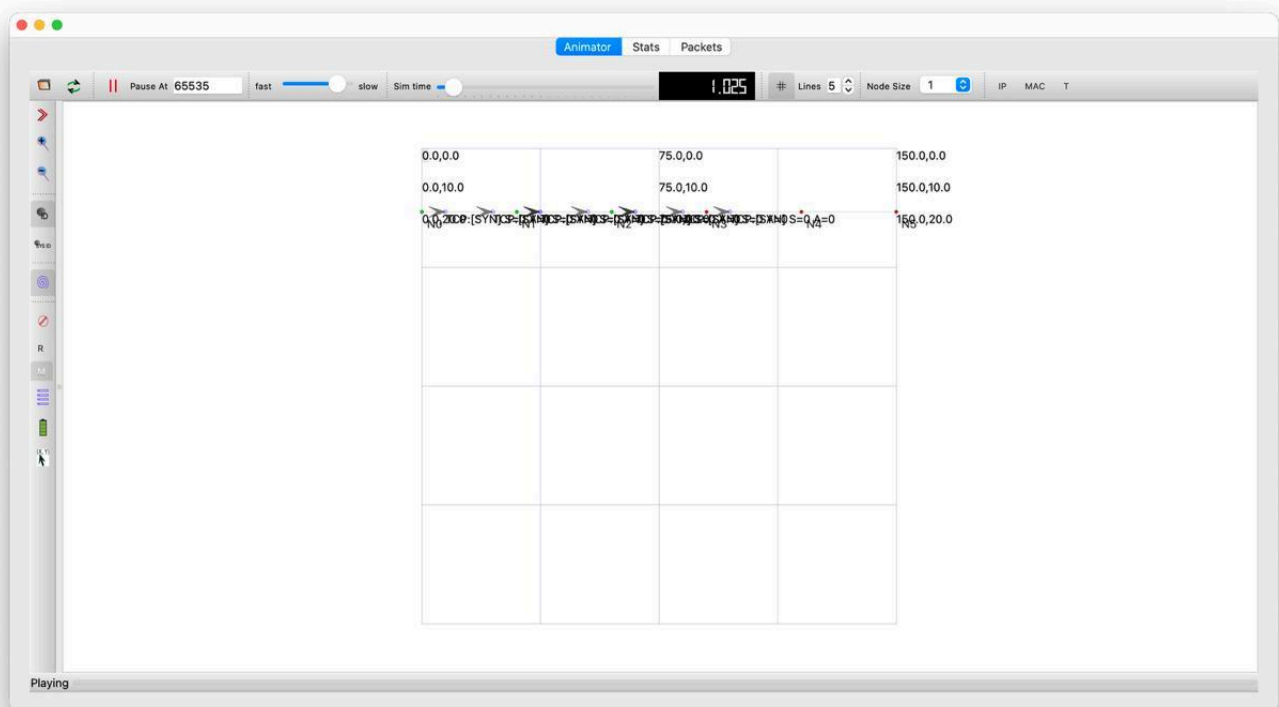
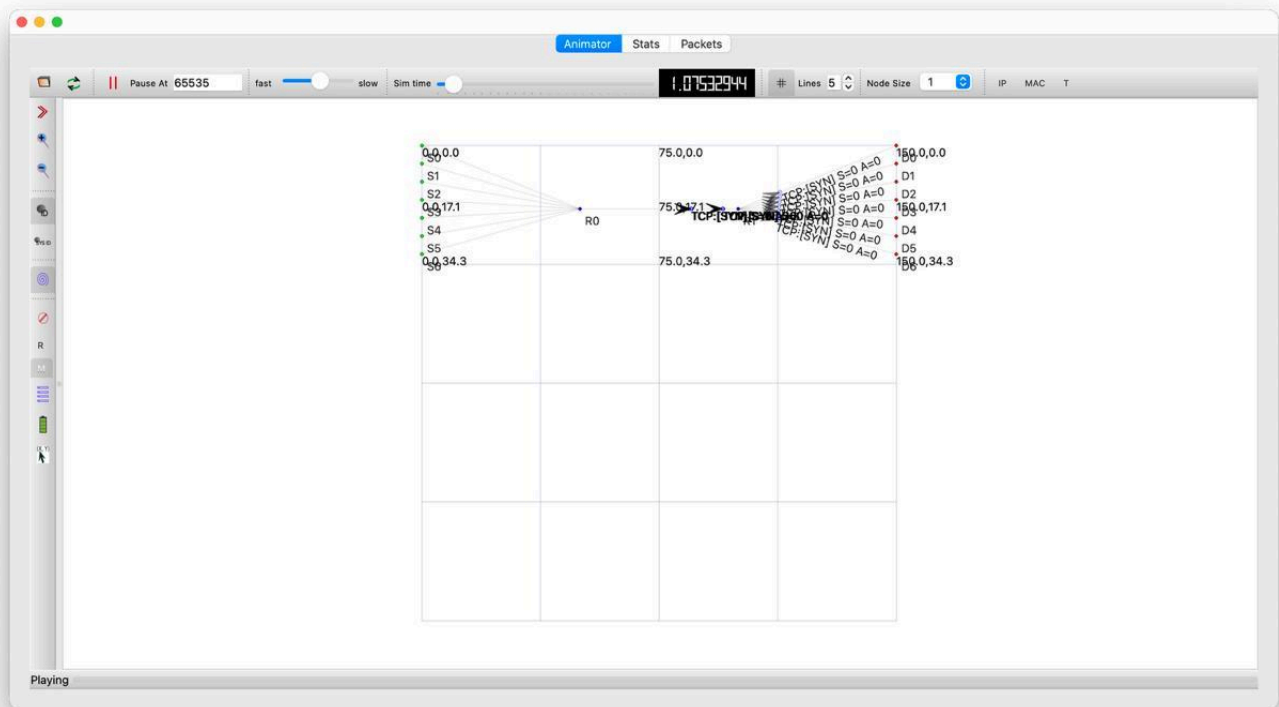


همچنین تعداد زیادی دیگر نمودار برای تحلیل وجود دارد اما ما به دلیل طولانی شدن زیاد گزارش آن‌ها را قرار ندادیم اما قابل مشاهده هستند.

تصاویری از شبیه‌سازی تصویری فاز دوم

تصاویر شبیه‌سازی هر ۳ توپولوژی توسط netanim می‌توانید ببینید:





جمع‌بندی

بر اساس نمودارها و کد نتایج زیر حاصل شد:

1. برنده توپولوژی: Tree (درختی). به دلیل ساختار موازی در سمت گیرنده (Root)، دو برابر توپولوژی‌های دیگر گذردهی (Goodput) داشت و تاخیر کمتری ایجاد کرد.
 2. بازنده توپولوژی: Bus (اتوبوسی). عبور از صف‌های متوالی باعث شد نرخ اتلاف Cubic به اوج برسد و تاخیر برای همه پروتکل‌ها زیاد شود.
 3. برنده پروتکل در ۲۰ جریان: TcpVegas. در تمام نمودارهای راداری، Vegas متعادل‌ترین عملکرد را داشت (اتلاف صفر، تاخیر کم، گذردهی مشابه بقیه).
- طبق فاز ۱، اگر تعداد جریان‌ها را به ۵۰ برسانیم، ورق برمی‌گردد. اما در این سناریوی خاص (۲۰ جریان)، Vegas بهترین است.